Dossier de projet OG Game Challenge

Titre professionnel Développeur web et web mobile Niveau 5

Application web full stack et plateforme de jeux

Candidat OUAHES Hédy

Formation Foundations de Holberton School

Session du titre professionnel 16 novembre 2026

Dossier de projet présenté au jury

Sommaire

1 Introduction

2 Expression des besoins

3 Organisation et conduite du projet

4 Environnement technique et architecture

5 Conception de l'interface et réalisation du frontend

6 Frontend dynamique et intégration des jeux

7 Base de données composants backend et sécurité

8 Tests

9 Installation et déploiement

10 Veille sur les vulnérabilités

11 Bilan personnel et conclusion

OG Game Challenge — contexte, besoins et organisation

1. Introduction

OG Game Challenge est une application web full-stack permettant à des utilisateurs de créer un compte, de jouer à plusieurs jeux directement depuis leur navigateur, d'enregistrer leurs scores et de consulter des classements.

Le projet a été réalisé dans le cadre du projet de fin de formation du programme Foundations de Holberton School. Cette formation de neuf mois propose un parcours progressif allant des langages de bas niveau jusqu'au développement web. Pour ce projet final, le thème était libre à condition de concevoir et de réaliser une application web fonctionnelle.

J'ai proposé le concept d'OG Game Challenge et assuré le rôle de chef de projet pendant la phase de développement en équipe. Mon objectif était de dépasser la réalisation d'un site vitrine en construisant une application complète réunissant une interface utilisateur, une API REST, une base de données relationnelle, une authentification et plusieurs jeux JavaScript.

Le projet devait également constituer une démonstration concrète de nos compétences auprès des autres étudiants et des recruteurs. Le choix d'une plateforme de jeux permettait de rendre cette démonstration interactive : au lieu de présenter uniquement du code ou des pages statiques, un utilisateur pouvait suivre un parcours complet, depuis son inscription jusqu'à l'apparition de son score dans un classement.

La première version a été développée en équipe du 20 avril au 6 juillet 2026. Après cette période, j'ai repris le projet pour réunir le frontend et le backend, consolider leur fonctionnement et poursuivre les améliorations du portfolio. J'ai ensuite conçu et développé Space Runner afin de compléter le catalogue avec un jeu plus personnel et plus ambitieux sur le plan technique.

1.1 Compétences mobilisées

Le projet permet de présenter les compétences suivantes du titre Développeur web et web mobile :

- maquetter des interfaces utilisateur web ou web mobile ;
- réaliser des interfaces utilisateur statiques web ou web mobile ;
- développer la partie dynamique des interfaces utilisateur ;
- mettre en place une base de données relationnelle ;
- développer des composants d'accès aux données ;
- développer des composants métier côté serveur.

Il permet également d'expliquer l'installation de l'environnement de travail et la documentation du lancement d'une application dynamique.

2. Expression des besoins

2.1 Idée initiale

L'idée d'OG Game Challenge est née de la volonté de créer un projet de fin de formation à la fois technique, ludique et facilement démontrable. Une plateforme de jeux permettait de réunir plusieurs domaines abordés pendant la formation : développement d'interfaces, programmation JavaScript, communication HTTP, logique métier, persistance des données et sécurité.

Le projet ne répondait pas à la commande d'une entreprise réelle. J'ai donc formulé le besoin et défini le périmètre fonctionnel avec l'équipe. Le produit devait rester réalisable dans le temps disponible tout en couvrant un parcours utilisateur complet.

2.2 Public visé

La plateforme visait principalement deux catégories d'utilisateurs :

- les autres étudiants, qui pouvaient découvrir les jeux, comparer leurs scores et participer à un défi commun ;
- les recruteurs, pour lesquels l'application servait de support concret afin d'observer les compétences techniques mobilisées dans un projet full-stack.

Ce double public a influencé les choix du projet. L'application devait être simple à comprendre et immédiatement utilisable, mais son architecture devait également être suffisamment structurée pour être expliquée lors d'une présentation technique.

2.3 Besoins fonctionnels

Le parcours principal devait permettre à un utilisateur de :

1. accéder à la page d'accueil ;

2. créer un compte avec un nom d'utilisateur, une adresse électronique et un mot de passe ;

3. se connecter et conserver une session ;

4. consulter le catalogue des jeux disponibles ;

5. lancer un jeu dans le navigateur ;

6. obtenir un score à la fin d'une partie ;

7. transmettre ce score au serveur ;

8. consulter le classement correspondant au jeu.

L'application devait donc proposer au minimum :

- une gestion des utilisateurs ;
- une authentification ;
- un catalogue de jeux ;
- plusieurs jeux JavaScript jouables dans le navigateur ;
- une sauvegarde persistante des scores ;
- un classement par jeu ;
- une navigation cohérente entre les différentes pages.

2.4 Besoins techniques

Pour répondre à ces fonctionnalités, l'application devait être séparée en plusieurs composants :

- un frontend en HTML, CSS et JavaScript pour l'affichage et les interactions ;
- une API REST chargée de recevoir les requêtes du navigateur ;
- une couche métier chargée de valider les opérations ;
- une base relationnelle pour conserver les utilisateurs, les jeux et les scores ;
- des jeux indépendants capables de transmettre leur résultat à l'application.

Cette séparation devait permettre aux membres de l'équipe de travailler en parallèle, puis de connecter leurs productions lors de la phase d'intégration.

2.5 Contraintes

La contrainte principale était le délai : la première version devait être développée entre le 20 avril et le 6 juillet 2026. Le projet devait donc privilégier un produit minimum viable cohérent plutôt qu'une accumulation de fonctionnalités.

Le thème était libre, mais le résultat attendu devait rester une application web. Nous avons également choisi de travailler sans framework frontend afin de mettre en pratique directement HTML, CSS et JavaScript. Le backend devait exposer des routes clairement documentées afin de faciliter la connexion avec le frontend et les jeux.

Le travail en équipe imposait une autre contrainte : les trois parties principales devaient pouvoir évoluer séparément sans empêcher leur intégration. Il était nécessaire de définir les responsabilités de chacun et les données échangées entre les composants.

2.6 Périmètre du MVP

Pour respecter le délai, les fonctionnalités ont été classées selon leur importance.

Les éléments indispensables étaient :

- l'inscription et l'authentification ;
- l'API REST ;
- la base de données ;
- le frontend ;
- les jeux jouables ;
- la transmission et l'affichage des scores.

Le responsive, la documentation Swagger et l'amélioration du classement constituaient des objectifs importants, mais secondaires par rapport au fonctionnement du parcours principal.

Certaines évolutions ont volontairement été exclues de la première version :

- le multijoueur en temps réel ;
- les succès et récompenses ;
- plusieurs modes de jeu ;
- le déploiement public complet ;
- une infrastructure cloud ;
- l'adaptation tactile complète des jeux.

Ces limites ont permis de concentrer le développement sur l'intégration entre le frontend, l'API, la base de données et les jeux.

2.7 Critères de réussite

Le MVP pouvait être considéré comme fonctionnel si un utilisateur était capable de créer un compte, de s'authentifier, de jouer, d'envoyer un score et de retrouver ce résultat dans le classement sans intervention manuelle dans la base de données.

La réussite technique dépendait également de la séparation des responsabilités : le frontend ne devait pas accéder directement à la base et les jeux ne devaient pas contenir la logique d'authentification ou de persistance.

3. Organisation et conduite du projet

3.1 Rôle de chef de projet

J'ai créé l'idée d'OG Game Challenge et organisé le travail de l'équipe. Mon rôle consistait à transformer l'idée générale en blocs réalisables, à répartir les responsabilités et à préciser les résultats attendus pour chaque partie.

La répartition initiale était la suivante :

Membre	Responsabilité principale
Hedy	Direction du projet, backend, API, base de données, authentification et logique métier
Nael	Développement des premiers jeux JavaScript
Geoffray	Développement de l'interface frontend

Cette répartition permettait un développement parallèle. Mon rôle ne se limitait toutefois pas à produire le backend : je devais également veiller à ce que les interfaces entre les trois blocs soient suffisamment précises pour permettre leur assemblage.

3.2 Découpage en deux phases

Le travail a été organisé en deux grandes phases.

La première phase était consacrée au développement indépendant :

- création de l'API, des modèles et de l'authentification ;
- création des pages et de l'identité graphique ;
- développement des jeux JavaScript ;
- définition du format des données échangées.

La seconde phase était consacrée à l'intégration :

- connexion des formulaires à l'API ;
- gestion de la session utilisateur ;
- intégration des jeux dans le portfolio ;
- transmission des scores ;
- récupération des classements ;
- correction des incompatibilités entre les composants.

Cette organisation réduisait les conflits au début du projet, mais rendait la phase d'intégration particulièrement importante. Un composant pouvait fonctionner isolément tout en rencontrant des difficultés une fois connecté aux autres.

3.3 Communication

L'équipe communiquait principalement de deux façons : en présentiel lorsque cela était possible et à distance sur Discord. Les échanges permettaient de préciser les tâches, de signaler les difficultés et de coordonner la réunion des différentes parties.

Le code était centralisé avec Git et GitHub. La séparation des responsabilités limitait initialement les modifications simultanées des mêmes fichiers. Lors de l'intégration, il a toutefois fallu harmoniser les arborescences, les routes et les données attendues par chaque composant.

3.4 Adaptation à une contrainte humaine

Pendant le projet, l'un des membres ne pouvait pas participer en présentiel. Cette situation risquait de créer un décalage entre son travail et les besoins des autres composants.

Pour maintenir sa participation, je lui ai proposé un plan de charge réalisable entièrement à distance. J'ai précisé les tâches à accomplir ainsi que les éléments attendus au moment de la livraison. Discord permettait de conserver un canal d'échange malgré l'absence physique.

Cette adaptation a permis de préserver la répartition du travail sans bloquer l'avancement général. Elle m'a également appris qu'une consigne technique ne doit pas seulement indiquer une fonctionnalité : elle doit préciser le résultat attendu, son périmètre et la manière dont il devra s'intégrer au reste du projet.

3.5 Intégration et poursuite individuelle

Après la période initiale en équipe, j'ai repris seul la fusion du backend et du frontend. Cette étape consistait à rendre cohérents des composants initialement développés séparément, puis à vérifier le parcours complet de l'utilisateur.

J'ai notamment travaillé sur :

- la connexion entre les formulaires et l'API ;
- l'utilisation du JWT dans le navigateur ;
- la transmission des scores ;
- l'affichage des classements ;
- l'intégration des jeux dans l'application ;
- l'harmonisation de l'arborescence et des routes.

J'ai ensuite développé seul Space Runner. Ce jeu représente la réalisation dont je suis le plus satisfait. Sa conception m'a permis d'approfondir la programmation orientée jeu vidéo, domaine vers lequel je souhaite désormais orienter mon parcours. Il a également enrichi le projet en apportant une réalisation plus ambitieuse tout en conservant l'intégration existante avec l'API de scores.

3.6 Premier bilan organisationnel

Cette organisation m'a surtout appris à préciser les contrats entre les composants et les résultats attendus avant de répartir le travail. La reprise individuelle m'a ensuite donné une vision complète du système et a préparé le développement de Space Runner.

4. Environnement technique et architecture

4.1 Des choix techniques déjà maîtrisés

J'ai choisi Python et Flask pour le backend. Ce choix n'était pas imposé par Holberton : il venait de mon expérience sur le projet HBnB, réalisé auparavant pendant la formation.

HBnB m'avait permis d'assimiler une architecture backend composée de plusieurs couches, avec une API, une façade, des repositories et des modèles. J'avais également déjà travaillé avec Flask, Flask-RESTX, SQLAlchemy et JWT. Pour OG Game Challenge, j'ai préféré réutiliser et adapter cette base plutôt que de consacrer le temps limité du projet à l'apprentissage d'un nouveau framework.

Ce choix avait deux avantages. Le premier était de réduire le risque technique : je connaissais déjà le rôle des outils et la manière dont ils pouvaient fonctionner ensemble. Le second était de pouvoir me concentrer sur les besoins propres au projet, notamment la gestion des scores, le classement et l'intégration des jeux.

Il ne s'agissait pas de recopier HBnB, mais de reprendre une organisation que je comprenais pour l'appliquer à un autre domaine. HBnB gérait des objets liés à une plateforme de location, tandis qu'OG Game Challenge devait gérer des utilisateurs, des jeux et leurs scores.

4.2 Technologies utilisées

Frontend

Le frontend repose sur HTML, CSS et JavaScript sans framework. HTML structure les pages, CSS porte l'identité visuelle et le responsive, tandis que JavaScript gère les formulaires, la session, les appels API, le catalogue, les classements et les jeux.

Cette solution reste légère et adaptée à la taille du projet. Elle permet également de montrer directement les mécanismes du navigateur sans les masquer derrière une bibliothèque supplémentaire.

Backend

Flask constitue le cœur de l'application serveur. Il initialise l'application, charge la configuration et reçoit les requêtes HTTP.

Flask-RESTX permet d'organiser l'API en namespaces et génère l'interface Swagger. Cette interface a été particulièrement utile pendant le développement, car elle permettait de tester chaque route avant même que le frontend soit connecté.

Les routes sont regroupées sous le préfixe /api/v1. Elles couvrent l'authentification, les utilisateurs, les jeux, les scores, les classements et le contrôle de santé de l'API.

Base de données

SQLAlchemy joue le rôle d'ORM, c'est-à-dire d'intermédiaire entre les objets Python et les tables relationnelles. Les modèles Python décrivent les utilisateurs, les jeux et les scores. SQLAlchemy génère ensuite les requêtes nécessaires pour dialoguer avec SQLite.

SQLite a été retenu pour le développement local. Il ne demande pas l'installation d'un serveur de base de données et reste suffisant pour un MVP exécuté sur un seul ordinateur. Ce choix simplifiait donc la mise en route du projet sans empêcher la démonstration des relations, des clés étrangères et des requêtes métier.

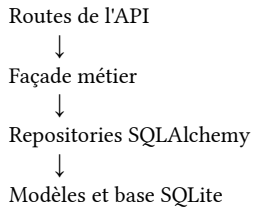
Authentification

L'authentification utilise des tokens JWT. Après une inscription ou une connexion valide, l'API renvoie un token au navigateur. Ce token accompagne ensuite les requêtes protégées.

Les mots de passe ne sont jamais enregistrés directement. Bcrypt produit une empreinte qui est conservée dans la base et utilisée lors de la vérification du mot de passe.

4.3 Architecture en couches

Le backend est organisé en quatre niveaux principaux :



Cette architecture est inspirée de celle étudiée dans HBnB. Je l'ai conservée parce qu'elle évite de mélanger toutes les responsabilités dans les routes.

Les routes reçoivent les données HTTP et construisent la réponse. Elles ne doivent pas contenir toute la logique du projet.

La façade applique les règles métier. Elle vérifie par exemple qu'un utilisateur n'existe pas déjà, authentifie un compte ou décide si un nouveau score doit remplacer l'ancien.

Les repositories regroupent les opérations génériques de persistance, comme l'ajout, la lecture ou la suppression d'un objet.

Les modèles décrivent enfin les données stockées et leurs relations.

Cette séparation rend le chemin d'une requête un peu plus long, mais elle facilite la lecture et les tests. Lorsqu'un problème apparaît, il est possible de déterminer s'il vient de la route, de la règle métier ou de la persistance.

4.4 Architecture générale de l'application

Le navigateur ne communique jamais directement avec SQLite. Toutes les opérations passent par l'API.

Lorsqu'un utilisateur réalise une action, le parcours général est le suivant :

1. l'utilisateur agit sur une page ou dans un jeu ;

2. JavaScript prépare une requête avec api.js ;

3. Flask reçoit la requête sur une route de l'API ;

4. la façade vérifie les données et applique la règle métier ;

5. SQLAlchemy lit ou modifie la base SQLite ;

6. l'API renvoie une réponse JSON ;

7. JavaScript met l'interface à jour.

Les jeux restent eux aussi séparés de la base. Ils calculent un résultat, puis utilisent une couche d'intégration pour le transmettre à l'API. Cette organisation permet de modifier la logique d'un jeu sans devoir réécrire le backend.

4.5 Le cas particulier des scores

La fusion du frontend et du backend n'a pas provoqué de blocage majeur. Les différentes parties avaient été pensées pour communiquer par l'API, ce qui a rendu leur connexion assez naturelle.

La partie la plus subtile concernait les scores. Il ne suffisait pas d'ajouter chaque résultat dans la base : l'objectif du classement était de conserver le meilleur score d'un utilisateur pour un jeu donné.

La façade commence donc par rechercher un score correspondant au même utilisateur et au même jeu. Si aucun score n'existe, elle crée une nouvelle entrée. Si un résultat existe déjà, elle compare les deux valeurs. Le score enregistré n'est remplacé que lorsque la nouvelle valeur est supérieure.

Le principe paraît simple, mais il est important pour la cohérence du classement. Sans cette comparaison, un joueur pourrait perdre sa meilleure performance en réalisant ensuite une partie moins bonne. À l'inverse, enregistrer toutes les tentatives multiplierait les lignes et pourrait faire apparaître plusieurs fois le même joueur dans le classement.

Le tri décroissant est ensuite appliqué lors de la récupération du classement afin d'afficher en premier les valeurs les plus élevées.

4.6 Organisation des dossiers

Le repository Git sépare les trois grandes parties du projet :

```
og-game-challenge/
├── backend/
│   ├── app/api/
│   ├── app/models/
│   ├── app/repositories/
│   ├── app/services/
│   └── tests/
├── frontend/
│   ├── assets/
│   ├── css/
│   ├── html/
│   └── js/
└── docs/
```

Cette arborescence correspond à la séparation choisie pendant le développement. Le frontend contient ce qui est exécuté ou affiché dans le navigateur. Le backend contient l'API, les règles métier, les modèles et les tests. Le dossier de documentation conserve les rapports et les diagrammes techniques.

Space Runner possède lui-même une organisation modulaire. Le jeu ayant pris plus d'ampleur que les premiers prototypes, séparer ses responsabilités est devenu nécessaire pour éviter un fichier JavaScript unique difficile à maintenir.

4.7 Recul sur les choix techniques

Avec du recul, je ne changerais pas profondément les technologies choisies. Elles correspondaient à ce que je maîtrisais et au délai disponible. Utiliser Flask, SQLAlchemy et SQLite m'a permis d'obtenir une application complète sans ajouter une infrastructure disproportionnée.

Si j'avais disposé de davantage de temps libre, j'aurais surtout consacré plus d'énergie à Space Runner. C'est la partie dans laquelle j'ai le plus progressé et celle qui correspond le mieux à mon orientation actuelle vers le jeu vidéo.

Je modifierais toutefois la manière de planifier certaines fonctionnalités du jeu. L'audio a été ajouté tardivement alors qu'il affecte de nombreux événements : lancement, tirs, ennemis, boss, transitions et écran final. L'intégrer dès la conception aurait évité plusieurs ajustements en fin de développement.

Les deux autres difficultés importantes de Space Runner ont été les performances et l'équilibrage. Il fallait conserver suffisamment d'éléments à l'écran pour donner du rythme sans rendre la boucle de jeu instable. Il fallait également augmenter progressivement la difficulté sans rendre les premières minutes trop faciles ou les dernières phases injustes.

5. Conception de l'interface et réalisation du frontend

5.1 Une identité liée au jeu vidéo

OG Game Challenge devait être identifiable dès la page d'accueil. L'objectif n'était pas de créer une interface neutre, mais de faire comprendre immédiatement que l'utilisateur entrait dans un univers de jeux et de compétition.

J'ai proposé l'idée générale du projet ainsi que son orientation. L'identité visuelle a ensuite été définie en commun avec Geoffray, responsable principal du frontend. Le résultat associe un univers arcade à une esthétique cyberpunk : fond sombre, violet lumineux, touches dorées, typographies géométriques et éléments qui rappellent un affichage de jeu.

J'ai particulièrement participé au design de la page d'accueil. Cette page devait rester assez simple pour ne pas perdre l'utilisateur : le nom du projet occupe l'espace principal, la navigation reste visible et un bouton invite directement à rejoindre la plateforme.

Le frontend a ensuite conservé cette même identité sur les pages d'inscription, de connexion, de catalogue et de classement. Cette cohérence était importante, car les jeux possèdent chacun leur propre style. L'interface du portfolio devait donc servir de cadre commun entre des expériences différentes.

5.2 Une interface rédigée en anglais

Les textes de l'interface sont principalement en anglais. Ce choix vient d'abord de mon habitude de développement : je considère l'anglais comme la langue commune du domaine informatique. Les noms de routes, les variables, les messages techniques et une grande partie des ressources utilisées pendant le développement sont déjà en anglais.

Dans le contexte du projet, cette langue correspondait également bien à l'identité arcade. Des termes comme « Challenge », « Play » ou « Leaderboard » sont courts, reconnaissables et cohérents avec les références du jeu vidéo.

Le dossier et la présentation restent rédigés en français afin d'expliquer clairement les choix techniques. L'anglais est donc un choix d'interface, pas une difficulté à traduire le fonctionnement du projet.

5.3 Parcours utilisateur

Le parcours principal est volontairement direct :

Accueil
↓
Inscription ou connexion
↓
Catalogue des jeux
↓
Partie
↓
Enregistrement du score
↓
Classement

La barre de navigation donne accès à l'accueil, aux défis et au classement. L'état de la session modifie la zone d'authentification afin d'afficher les actions adaptées à un visiteur ou à un utilisateur connecté.

Le catalogue présente chaque jeu dans une carte avec deux actions principales : lancer le jeu ou consulter son classement. L'utilisateur n'a donc pas besoin de parcourir plusieurs écrans avant de comprendre ce qu'il peut faire.

5.4 Maquettes reconstituées

Aucune maquette source réalisée avant le développement n'a été conservée. Pour documenter la démarche sans prétendre le contraire, deux maquettes filaires ont été reconstituées à partir du résultat final.

Le catalogue a été retenu comme écran représentatif, car il réunit la navigation, le titre, la liste des jeux et les actions principales. Une version desktop présente les cartes sur plusieurs colonnes. Une version mobile place les cartes les unes sous les autres et réorganise la navigation.

Ces documents ne sont pas présentés comme des archives du début du projet. Ils servent à formaliser les choix d'organisation et à montrer comment une même interface s'adapte à deux tailles d'écran. Les captures du site réel constituent une preuve séparée du résultat obtenu.

5.5 Structure du frontend

Le frontend utilise HTML, CSS et JavaScript sans framework. Les fichiers sont répartis par responsabilité :

```
frontend/
├── html/      pages du site et jeux intégrés
├── css/       styles globaux, styles des pages et responsive
├── js/        session, appels API, interfaces dynamiques et jeux
└── assets/    images, sons, vidéos et ressources des jeux
```

Chaque page HTML charge une feuille globale, une feuille correspondant à son interface et la couche responsive commune. Cette organisation permet de partager la navigation, les couleurs et les typographies tout en conservant des règles spécifiques pour l'accueil, les formulaires, le catalogue ou le classement.

Les variables CSS définissent les couleurs principales et les polices. Cela évite de répéter partout les mêmes valeurs et facilite une modification globale de l'identité visuelle.

5.6 Pages statiques

Les interfaces sont structurées avec des éléments HTML simples : navigation, contenu principal, formulaires, cartes, boutons et tableau de classement.

La page d'accueil joue le rôle d'entrée dans le projet. Les pages d'inscription et de connexion utilisent des formulaires lisibles avec des messages de retour. Le catalogue réserve une grille dans laquelle les jeux sont ajoutés par JavaScript. Le classement contient des onglets et une zone destinée aux scores récupérés depuis l'API.

Même lorsque leur contenu devient dynamique, ces pages reposent d'abord sur une structure statique claire. JavaScript complète ensuite les zones prévues sans reconstruire toute la page.

5.7 Adaptation mobile récente

Le responsive faisait partie de l'intention générale du projet, mais l'adaptation mobile visible aujourd'hui a surtout été renforcée récemment. Il est important de conserver cette chronologie dans le dossier.

Une feuille responsive.css commune a été ajoutée après les styles propres à chaque page. Elle adapte notamment :

- la navigation, qui peut passer sur deux lignes ;
- la taille des titres ;
- les marges latérales ;
- les cartes de jeux, affichées sur une colonne ;

- les boutons, qui utilisent davantage la largeur disponible ;
- les colonnes du classement ;
- la taille des zones Canvas.

Cette couche commune évite de répéter les mêmes media queries dans chaque feuille de style. Elle permet aujourd'hui de consulter l'accueil, les formulaires, le catalogue et le classement depuis un téléphone connecté au même réseau local que l'ordinateur.

Les quatre captures réalisées sur téléphone montrent que ces interfaces restent lisibles et utilisables. Les jeux, en revanche, n'ont pas encore tous de commandes tactiles. Le responsive du site et la jouabilité tactile sont deux sujets différents : le premier est pris en charge, tandis que le second reste une évolution possible.

5.8 Répartition du travail sur le frontend et les jeux

Geoffray a assuré la réalisation principale du frontend. L'identité générale a été décidée en commun, et j'ai participé plus directement au design de la page d'accueil. Mon rôle de chef de projet consistait également à vérifier que les interfaces prévues permettaient d'accéder aux fonctions nécessaires : inscription, connexion, catalogue, jeux et classement.

Pour les deux premiers jeux, j'ai défini le type de jeu attendu. Nael a ensuite conçu et développé lui-même Dodge Runner et Platformer Escape. Cette répartition lui laissait une autonomie sur les mécaniques et leur réalisation tout en maintenant une direction commune pour la plateforme.

Space Runner a été conçu et développé entièrement par moi après la phase initiale. Il reprend le même principe d'intégration : le jeu reste autonome pour son gameplay, mais transmet son résultat à l'API du portfolio.

Cette distinction permet de présenter clairement la répartition du travail. Le dossier rend compte de l'application dans son ensemble tout en identifiant précisément ma contribution et celle de chaque membre.

5.9 Premiers éléments d'accessibilité

Le projet ne prétend pas atteindre une conformité complète à un référentiel d'accessibilité. Plusieurs choix améliorent néanmoins l'utilisation :

- contraste important entre le fond sombre et les textes ;
- états visuels sur les liens actifs et les boutons ;
- adaptation de la taille des éléments sur petit écran ;
- indication des commandes de Space Runner ;
- compatibilité AZERTY et QWERTY pour ce jeu ;
- limitation possible de certains effets lorsque l'utilisateur réduit les animations.

Ces éléments restent des améliorations ciblées. Une vérification complète au clavier, avec lecteur d'écran et outils spécialisés constituerait une étape supplémentaire.

5.10 Bilan de la conception frontend

L'interface fournit un cadre cohérent aux trois jeux et au parcours utilisateur. L'ajout récent de la couche responsive prolonge cette conception sans modifier l'identité visuelle initiale.

6. Frontend dynamique et intégration des jeux

6.1 Une interface reliée à l'API

Les pages HTML donnent une structure à l'application, mais une grande partie de leur contenu dépend du backend. Le catalogue, la session et les classements ne sont pas écrits directement dans les pages : JavaScript les construit à partir des réponses de l'API.

J'ai centralisé cette communication dans api.js. Ce fichier définit l'adresse de l'API, ajoute les en-têtes nécessaires, exécute les requêtes avec fetch() et transforme les erreurs HTTP en erreurs utilisables par l'interface.

L'adresse n'est plus limitée à 127.0.0.1. Elle reprend le protocole et le nom d'hôte utilisés pour ouvrir le frontend, puis ajoute le port 5001. Cette adaptation permet au même code de fonctionner depuis l'ordinateur et depuis un téléphone connecté au réseau local.

Les autres fichiers n'ont pas besoin de reconstruire une requête complète. Ils utilisent des fonctions comme register(), login(), getGames(), submitScore() ou getLeaderboard(). Cette centralisation évite de répéter l'adresse de l'API et le traitement des réponses dans chaque page.

6.2 Gestion de la session

Après une inscription ou une connexion réussie, le frontend conserve le JWT et les informations publiques de l'utilisateur dans le stockage local du navigateur.

Le fichier session.js utilise ces données pour adapter la navigation. Un visiteur voit les actions d'inscription et de connexion. Un utilisateur connecté voit son nom et un bouton de déconnexion. La page d'accueil masque également l'appel à l'inscription lorsqu'une session existe déjà.

Le JWT est ajouté automatiquement dans l'en-tête Authorization des requêtes suivantes :

```
Authorization: Bearer <token>
```

Le frontend sait donc transmettre la preuve d'authentification, mais il ne décide pas si l'action est autorisée. Cette décision reste prise par le backend.

Le stockage local est simple et adapté à ce MVP, mais il présente une limite : un script injecté dans la page pourrait tenter d'y lire le token. Une version de production demanderait une réflexion plus poussée sur la protection contre les attaques XSS et sur l'utilisation éventuelle de cookies sécurisés.

6.3 Catalogue dynamique

La page du catalogue contient une grille vide au chargement. Le fichier games.js appelle GET /games/, puis crée une carte pour chaque jeu renvoyé par l'API.

Les informations métier, comme le nom et la description, viennent de la base. Les éléments propres à l'interface, comme la vidéo de prévisualisation, l'image et la page à ouvrir, restent définis dans le frontend.

Cette séparation évite de placer des chemins de fichiers dans la base de données. Elle permet aussi d'afficher automatiquement les jeux enregistrés tout en gardant le contrôle sur leur présentation.

Si l'API n'est pas disponible, la page affiche un message compréhensible au lieu de rester vide. Les prévisualisations vidéo se lancent au survol et reviennent au début lorsque le pointeur quitte la carte.

6.4 Classement dynamique

La page de classement récupère d'abord la liste des jeux pour construire les onglets. Lorsqu'un onglet est sélectionné, elle appelle la route de classement correspondante.

Le backend renvoie les scores déjà triés et attribue un rang. Le frontend se limite à afficher les dix premières entrées et à rechercher la position de l'utilisateur connecté.

Cette répartition est volontaire. Le classement constitue une règle liée aux données et doit rester cohérent quel que soit le client. Le navigateur ne recalcule donc pas l'ordre à partir d'une liste non triée : il utilise le résultat préparé par l'API.

6.5 Adaptateur commun pour les scores

Les trois jeux possèdent des mécaniques différentes, mais leur fin de partie doit produire le même résultat pour le portfolio. Le fichier score-adapter.js sert d'intermédiaire commun.

Lorsqu'une partie se termine, l'adaptateur :

1. vérifie qu'un utilisateur est connecté ;

2. identifie le jeu grâce à l'URL ;

3. récupère son identifiant dans l'API ;

4. transmet le score ;

5. reçoit le meilleur score conservé par le backend ;

6. affiche un écran de fin adapté.

Le jeu ne connaît donc pas la structure de la base ni la logique de classement. Il transmet seulement son résultat. Cette séparation a facilité l'ajout de Space Runner, car il a pu utiliser le même contrat que les jeux existants.

6.6 Space Runner

Space Runner est un shoot'em up spatial développé en JavaScript avec Canvas 2D, sans moteur de jeu externe. Le joueur traverse plusieurs phases, affronte différents ennemis, récupère des améliorations et termine sa progression par trois boss.

Le jeu repose sur une boucle avec requestAnimationFrame(). À chaque image, une phase met à jour les déplacements, les collisions, les projectiles et les états. Une seconde phase dessine le résultat dans le Canvas.

J'ai séparé le jeu en modules pour éviter un seul fichier de plusieurs milliers de lignes. Le joueur, les ennemis, les météores, les bonus, les collisions, les boss, les projectiles et l'audio possèdent chacun leur propre responsabilité.

Cette organisation a été nécessaire au fur et à mesure que le jeu grandissait. Les boss disposent de comportements différents, et le dragon final utilise plusieurs phases. Un découpage modulaire permettait de corriger une mécanique sans risquer de modifier toutes les autres.

6.7 Performances, audio et équilibrage

Les performances ont représenté l'une des difficultés principales. Le jeu peut afficher simultanément des ennemis, des projectiles, des météores, des effets et un boss. Il fallait nettoyer les objets devenus inutiles et stabiliser la boucle pour éviter une dégradation progressive.

L'audio a été ajouté tardivement. Ce choix a demandé plus de travail que prévu, car les sons dépendent de nombreux événements et doivent être arrêtés, repris ou remplacés au bon moment. Si je recommençais cette partie, je définirais le fonctionnement audio plus tôt.

L'équilibrage ne pouvait pas être résolu par une formule unique. J'ai dû ajuster la fréquence des ennemis, les dégâts, les bonus et les attaques des boss pour construire une progression exigeante sans la rendre injuste.

6.8 Limites actuelles

Le frontend dynamique remplit les objectifs du MVP, mais plusieurs limites restent connues :

- le token est conservé dans localStorage ;
- les jeux sont principalement conçus pour le clavier ;
- le site dépend d'une API lancée localement ;
- le frontend associe encore les jeux à leurs ressources à partir de leur nom ;
- aucun test automatisé du navigateur n'est encore présent.

Ces limites ne remettent pas en cause le parcours démontré. Elles permettent surtout de distinguer un MVP fonctionnel d'une version destinée à une mise en production publique.

7. Base de données, composants backend et sécurité

7.1 Modèle relationnel

La base repose sur trois entités principales : l'utilisateur, le jeu et le score.

Un utilisateur peut obtenir plusieurs scores, mais chaque score appartient à un seul utilisateur. Un jeu peut recevoir plusieurs scores, mais chaque score concerne un seul jeu. La table scores relie donc users et games grâce à deux clés étrangères.

Les trois tables héritent d'un modèle de base qui ajoute un identifiant UUID, une date de création et une date de modification. Les UUID sont stockés sous forme de chaînes de 36 caractères.

La table users contient un email unique, un nom d'utilisateur unique et une empreinte Bcrypt du mot de passe. La table games contient un nom et une description. La table scores contient la valeur obtenue et les références vers l'utilisateur et le jeu.

Le MCD présente cette logique indépendamment de SQL. Le MLD la traduit ensuite en tables, clés primaires, clés étrangères, types et contraintes.

7.2 Choix de SQLite et SQLAlchemy

SQLite correspond au contexte local du projet. La base tient dans un fichier, ne demande pas de serveur supplémentaire et suffit pour tester les relations et les requêtes du MVP.

SQLAlchemy évite d'écrire toutes les requêtes SQL à la main. Les modèles Python sont associés aux tables et l'ORM construit les opérations nécessaires. Cela réduit aussi le risque d'injection SQL lorsque les filtres et paramètres sont utilisés correctement.

Ce choix ne signifie pas que la base est indépendante de toute réflexion relationnelle. Il reste nécessaire de définir les clés, les relations, les contraintes et les règles qui appartiennent au code métier.

7.3 Initialisation

Au lancement, SQLAlchemy crée les tables manquantes avec db.create_all(). Une fonction ajoute ensuite les trois jeux par défaut si la table est vide.

Cette solution est pratique pour une démonstration locale, car une nouvelle base peut être préparée automatiquement. Elle reste toutefois plus limitée qu'un système de migrations versionnées. Dans un projet en production, un outil comme Flask-Migrate permettrait de conserver l'historique des évolutions du schéma.

7.4 Repository générique

Le repository SQLAlchemy regroupe les opérations simples communes aux modèles : ajouter, lire par identifiant, lire la liste complète et supprimer.

Il reçoit la classe du modèle lors de sa création. La façade peut donc utiliser la même structure pour les utilisateurs, les jeux et les scores.

J'ai conservé cette couche parce qu'elle sépare la persistance de la logique métier. Une route n'a pas besoin de connaître db.session, et une règle métier n'a pas besoin de répéter les mêmes opérations de base.

Les requêtes plus spécifiques restent dans la façade lorsqu'elles expriment directement un besoin du projet, comme rechercher un utilisateur par email ou trier les scores d'un jeu.

7.5 Façade métier

La façade constitue le point central du backend. Elle utilise les repositories et les modèles, mais présente des méthodes correspondant aux actions de l'application.

Lors d'une inscription, elle vérifie d'abord que l'email et le nom d'utilisateur ne sont pas déjà utilisés. Elle crée ensuite l'utilisateur, dont le modèle hache le mot de passe avant l'enregistrement.

Lors d'une connexion, elle recherche l'utilisateur par son nom puis compare le mot de passe fourni à l'empreinte enregistrée.

Pour un score, elle vérifie l'existence de l'utilisateur et du jeu. Elle recherche ensuite un résultat existant pour ce couple. Si la nouvelle valeur est supérieure, elle remplace l'ancienne. Dans le cas contraire, elle conserve le meilleur résultat déjà enregistré.

Enfin, le classement filtre les scores par jeu et applique un tri décroissant sur leur valeur.

7.6 Une règle métier non remplacée par le frontend

La conservation du meilleur score doit rester côté serveur. Si la comparaison était réalisée uniquement dans JavaScript, un utilisateur pourrait contourner cette logique en appelant directement l'API.

Le frontend peut afficher le meilleur résultat reçu, mais il ne décide pas de la valeur conservée. Cette décision appartient à la façade, qui constitue la source de vérité.

La règle est actuellement garantie par le code métier. La base ne possède pas encore de contrainte unique composée sur user_id et game_id. Une telle contrainte renforcerait la cohérence en cas d'accès concurrents, mais elle demanderait aussi une migration du schéma.

7.7 Authentification et autorisation

Après une inscription ou une connexion valide, Flask-JWT-Extended crée un JWT dont l'identité correspond à l'UUID de l'utilisateur.

Les routes sensibles utilisent @jwt_required(). Pour la consultation d'un profil, le backend compare l'identité du token avec l'identifiant demandé. Un utilisateur ne peut donc pas lire le profil d'un autre compte en changeant simplement l'URL.

Pour l'enregistrement d'un score, l'API ignore toute identité éventuellement envoyée dans le corps de la requête. Elle utilise uniquement get_jwt_identity(). Cette correction est importante : sans elle, un utilisateur pourrait attribuer un score à un autre compte.

7.8 Protection des mots de passe et des secrets

Les mots de passe sont hachés avec Bcrypt. Le backend ne peut pas retrouver le mot de passe d'origine : il vérifie seulement si une valeur proposée correspond à l'empreinte.

Les clés Flask et JWT ne sont plus écrites dans le code. Elles sont chargées depuis un fichier .env ignoré par Git. Le fichier .env.example indique uniquement les noms des variables attendues.

Lors de la préparation du dossier, une capture Swagger a montré une faiblesse : la méthode générique de sérialisation renvoyait aussi password_hash dans la réponse utilisateur. Une empreinte n'est pas le mot de passe, mais elle ne doit jamais être exposée au client.

J'ai corrigé ce point en redéfinissant User.to_dict() pour retirer systématiquement password_hash. J'ai ensuite ajouté des assertions dans les tests d'inscription et de lecture du profil. Cette démarche a permis de transformer une observation de documentation en amélioration concrète de la sécurité.

7.9 Validation des données

Les modèles vérifient plusieurs règles avant l'enregistrement :

- l'email doit être une chaîne non vide contenant @ ;
- le nom d'utilisateur ne peut pas être vide ;
- le mot de passe ne peut pas être vide ;
- un score doit être un entier positif ou nul ;
- le jeu et l'utilisateur associés au score doivent exister.

Le frontend effectue aussi des contrôles pour améliorer l'expérience, comme la présence des champs et une longueur minimale du mot de passe. Les contrôles du navigateur ne remplacent toutefois jamais ceux du serveur.

7.10 Limites de sécurité connues

Le projet présente plusieurs limites que je préfère identifier clairement :

- CORS accepte actuellement toutes les origines ;
- le mode de développement utilise le serveur Flask intégré ;
- l'application locale n'utilise pas HTTPS ;
- le JWT est stocké dans localStorage ;
- la politique de mot de passe reste simple ;
- aucune limitation de fréquence n'est appliquée aux tentatives de connexion ;
- la base utilise create_all() plutôt que des migrations.

Ces éléments sont acceptables pour un MVP local, mais ils devraient être traités avant une ouverture publique. Le dossier distingue donc les protections réellement mises en œuvre des recommandations de production.

8. Tests, déploiement, veille et bilan

8.1 Stratégie de test

Pendant le développement initial, les tests étaient principalement manuels. J'ai utilisé Swagger pour appeler les routes une par une avant de connecter le frontend. Cette méthode permettait de vérifier les données envoyées, les codes HTTP et les réponses JSON.

Une fois les composants réunis, j'ai testé le parcours complet dans le navigateur : inscription, connexion, consultation du catalogue, lancement d'un jeu, envoi d'un score et apparition dans le classement.

Les jeux ont également été vérifiés séparément. Les contrôles portaient sur le lancement, les commandes, le calcul du score, la fin de partie, le redémarrage et la transmission du résultat.

Cette stratégie manuelle était adaptée à la première livraison, mais elle dépendait de manipulations répétées. Des tests automatisés ont donc été ajoutés pendant la consolidation du projet.

8.2 Tests automatisés

Le backend contient aujourd'hui sept tests d'intégration exécutés avec unittest. Chaque test démarre l'application avec une configuration dédiée et utilise une base SQLite temporaire.

Cette isolation évite de modifier la base utilisée pour la démonstration. Après chaque test, la session est fermée, les tables sont supprimées et le dossier temporaire est nettoyé.

Les tests couvrent :

1. le contrôle de santé et la liste des trois jeux ;

2. l'inscription, la connexion et la lecture de son propre profil ;

3. l'absence de l'ancienne route de création d'utilisateur ;

4. le refus d'une inscription dupliquée et d'un mot de passe incorrect ;

5. l'interdiction de consulter le profil d'un autre utilisateur ;

6. l'obligation d'utiliser un JWT et l'identité du token pour un score ;

7. la conservation du meilleur score lorsqu'une nouvelle valeur est inférieure.

Le 8 novembre 2026, les sept tests ont été exécutés après la correction de la sérialisation utilisateur. Ils ont tous réussi.

8.3 Jeu d'essai représentatif

La fonctionnalité choisie pour le jeu d'essai est la conservation du meilleur score. Elle traverse la route API, l'authentification, la façade, SQLAlchemy et la base de données.

Étape	Donnée en entrée	Résultat attendu	Résultat obtenu
Création du joueur	compte Alice	réponse 201 et JWT	conforme
Premier score	200 points	création du score à 200	conforme
Deuxième score	50 points	conservation de 200	conforme

Étape	Donnée en entrée	Résultat attendu	Résultat obtenu
Lecture du classement	identifiant du jeu	Alice classée avec 200	conforme

Le test vérifie explicitement que la seconde réponse contient toujours la valeur 200 et que le premier élément du classement possède également cette valeur.

Il n'y a donc pas d'écart entre le résultat attendu et le résultat obtenu. Ce cas démontre que la règle est appliquée par le backend et pas seulement par l'affichage du navigateur.

8.4 Tests de sécurité

Plusieurs tests vérifient des refus plutôt que des cas de réussite :

- une requête de profil sans JWT reçoit 401 ;
- un utilisateur authentifié demandant le profil d'un autre reçoit 403 ;
- un score sans JWT reçoit 401 ;
- un identifiant utilisateur ajouté au JSON ne remplace pas l'identité du JWT ;
- une inscription dupliquée reçoit 400 ;
- une connexion avec un mauvais mot de passe reçoit 401.

Après la découverte de l'exposition de password_hash, deux assertions supplémentaires vérifient que ce champ n'apparaît ni dans la réponse d'inscription ni dans celle du profil.

Ces contrôles permettent de conserver une preuve reproductible des corrections de sécurité.

8.5 Limites des tests

La suite actuelle se concentre sur l'intégration backend. Elle ne couvre pas encore automatiquement :

- les interactions réelles dans le navigateur ;
- le responsive ;
- les parties complètes des jeux ;
- les performances de Space Runner ;
- l'accessibilité ;
- les erreurs réseau entre le frontend et l'API.

Ces éléments ont été vérifiés manuellement. Si le projet continuait, des tests frontend et des scénarios de navigateur automatisés compléteraient la suite.

9. Installation et déploiement

9.1 Lancement local

L'application utilise deux serveurs locaux. Le backend Flask écoute sur le port 5001 et le serveur statique du frontend sur le port 8000.

Le lancement suit cet ordre :

1. créer le fichier .env à partir de .env.example ;

2. générer les deux secrets ;

3. installer les dépendances Python ;

4. lancer python3 run.py dans le backend ;

5. lancer python3 -m http.server 8000 --bind 0.0.0.0 dans le frontend ;

6. ouvrir la page d'accueil dans le navigateur.

Swagger est disponible à l'adresse locale /api/v1/. La route de santé permet de vérifier rapidement que l'API répond.

9.2 Accès depuis un téléphone

Le backend et le frontend écoutent sur 0.0.0.0. Un téléphone connecté au même Wi-Fi peut donc utiliser l'adresse privée de l'ordinateur.

Le JavaScript construit l'adresse de l'API à partir du nom d'hôte utilisé dans le navigateur. Si le site est ouvert avec 192.168.1.55, les appels sont automatiquement envoyés vers 192.168.1.55:5001.

Cette méthode a permis de vérifier réellement l'accueil, l'inscription, le catalogue et le classement sur téléphone. Elle ne transforme pas l'ordinateur en hébergement public : les deux serveurs doivent rester actifs et les appareils doivent partager le réseau local.

9.3 État réel du déploiement

Le portfolio complet n'est pas déployé sur un serveur public. GitHub Pages peut héberger des fichiers statiques, mais pas l'API Flask ni la base SQLite côté serveur.

Je préfère présenter clairement cette limite plutôt que de qualifier le lancement local de mise en production. La procédure actuelle constitue un déploiement local reproductible et une démonstration sur réseau privé.

9.4 Recommandations pour une production

Une mise en ligne demanderait au minimum :

- un serveur WSGI au lieu du serveur de développement Flask ;
- HTTPS ;
- des secrets propres à l'environnement de production ;
- une origine CORS limitée au domaine du frontend ;
- une base sauvegardée et, selon la charge, un serveur PostgreSQL ;
- des migrations versionnées ;
- des journaux et une surveillance du service ;
- une politique de mot de passe et une limitation des tentatives ;
- une stratégie de renouvellement ou d'expiration des sessions.

Il n'est pas nécessaire d'ajouter Docker ou Kubernetes pour démontrer cette compétence. La priorité est de savoir expliquer l'environnement actuel et les changements indispensables avant une ouverture publique.

10. Veille sur les vulnérabilités

10.1 Démarche

La veille présentée ici a été formalisée pendant la consolidation du projet pour le RNCP. Je ne la présente pas comme un document produit pendant la phase initiale de juin.

Je me suis appuyé sur l'OWASP Top 10 et sur les documentations des bibliothèques utilisées afin de comparer les risques courants aux choix réels du projet.

L'OWASP Top 10 2025 place notamment en avant le contrôle d'accès défaillant, les mauvaises configurations, les défaillances cryptographiques, les injections et les échecs d'authentification. Ces catégories correspondent directement à plusieurs points d'OG Game Challenge.

10.2 Contrôle d'accès

Le risque concernait principalement les profils et les scores. Une route ne doit pas se contenter de recevoir un identifiant fourni par le navigateur.

La lecture d'un profil compare maintenant l'identité JWT à l'identifiant demandé. La création d'un score utilise uniquement l'identité extraite du token. Les tests vérifient le refus des accès non authentifiés et des consultations d'un autre profil.

10.3 Données sensibles et cryptographie

Bcrypt protège les mots de passe enregistrés et les secrets sont sortis du code source. La revue a également permis de repérer l'exposition de password_hash dans une réponse JSON. Le champ a été supprimé des représentations publiques et un test empêche sa réapparition.

Le transport local reste en HTTP. Cette situation est acceptable sur la machine de développement, mais pas pour une authentification publique. Une production devrait obligatoirement utiliser HTTPS.

10.4 Injection et validation

Le backend utilise les filtres et requêtes paramétrées de SQLAlchemy au lieu de construire des chaînes SQL avec les valeurs reçues. Les modèles valident également le type et le contenu de plusieurs champs.

Cette protection ne dispense pas de vérifier toutes les entrées. Elle évite surtout la construction manuelle de requêtes contenant directement des données non fiables.

10.5 Mauvaise configuration

La configuration CORS actuelle accepte toutes les origines. Ce réglage facilite le développement local et l'accès depuis le téléphone, mais il est trop large pour une production.

Flask-CORS permet de limiter explicitement les origines autorisées. Une version publique devrait n'accepter que le domaine réel du frontend et séparer clairement les configurations de développement, de test et de production.

10.6 Résultat de la veille

Cette veille a conduit ou contribué à plusieurs actions concrètes :

- utilisation obligatoire du JWT sur les routes sensibles ;

- identité des scores prise dans le token ;
- protection du profil d'un autre utilisateur ;
- secrets chargés depuis .env ;
- suppression de password_hash des réponses publiques ;
- tests automatisés des refus d'accès ;
- identification de CORS, HTTPS et du stockage du token comme limites restantes.

10.7 Sources retenues

- OWASP Top 10 2025 : https://top10.owasp.org/2025/0x00_2025-Introduction/ ;
- OWASP Developer Guide : https://devguide.owasp.org/en/02-foundations/05-top-ten/ ;
- documentation Flask-CORS : https://flask-cors.readthedocs.io/en/latest/configuration.html ;
- documentation SQLAlchemy sur les paramètres liés : https://docs.sqlalchemy.org/en/13/faq/sqlexpressions.html.

11. Bilan personnel et conclusion

11.1 Résultat obtenu

OG Game Challenge remplit aujourd'hui le parcours principal imaginé au départ : un utilisateur peut créer un compte, se connecter, consulter trois jeux, jouer, enregistrer son meilleur score et consulter un classement. Le projet réunit un frontend, une API, une base relationnelle et une authentification, avec une interface utilisable sur ordinateur et consultable sur téléphone.

11.2 Ce que le projet m'a apporté

Au début, ma responsabilité principale concernait le backend. La reprise individuelle m'a obligé à comprendre l'ensemble du parcours, depuis le clic dans le navigateur jusqu'à la base de données.

Le rôle de chef de projet m'a aussi appris à répartir un besoin, préciser un résultat attendu et m'adapter à la disponibilité d'un membre. La difficulté n'était pas seulement d'écrire du code, mais de faire en sorte que plusieurs productions puissent fonctionner ensemble.

11.3 Space Runner et mon orientation

Space Runner reste la partie dont je suis le plus satisfait. Le développer entièrement m'a permis de travailler sur une boucle temps réel, les collisions, les états, les boss, l'audio, les performances et l'équilibrage.

Cette expérience a renforcé mon envie de m'orienter vers le jeu vidéo. Elle m'a montré que j'appréciais autant la construction des mécaniques et du ressenti que l'architecture générale d'une application.

11.4 Recul sur le projet

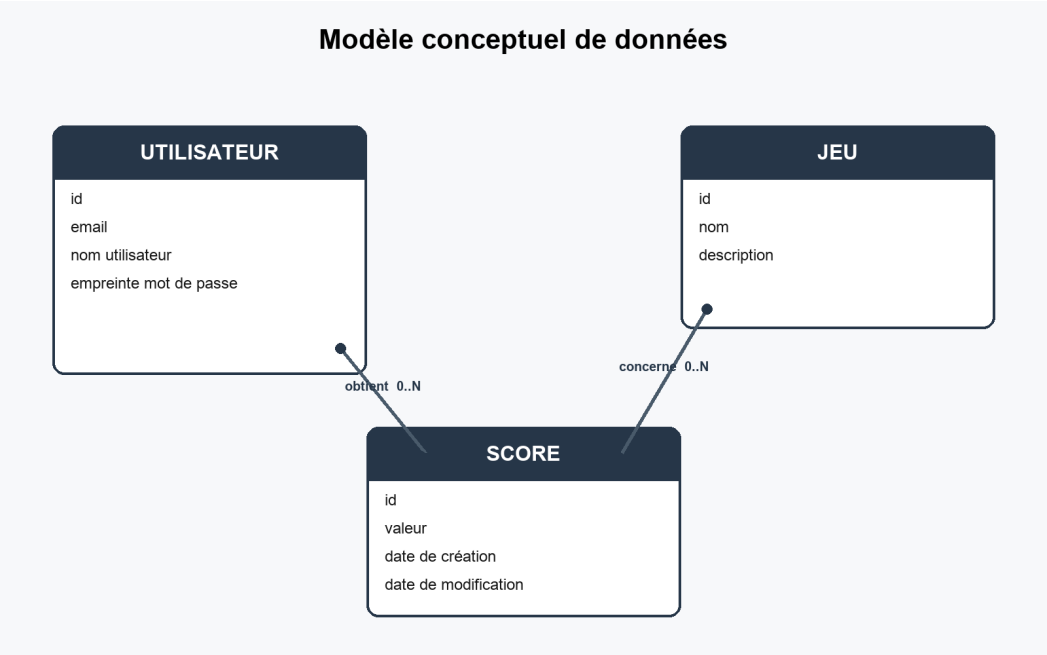
Avec davantage de temps, je prioriserais l'audio, les mesures de performance et les essais d'équilibrage de Space Runner. Sur la partie web, je renforcerais la configuration de production, les migrations, les tests frontend et la protection de la session.

11.5 Conclusion

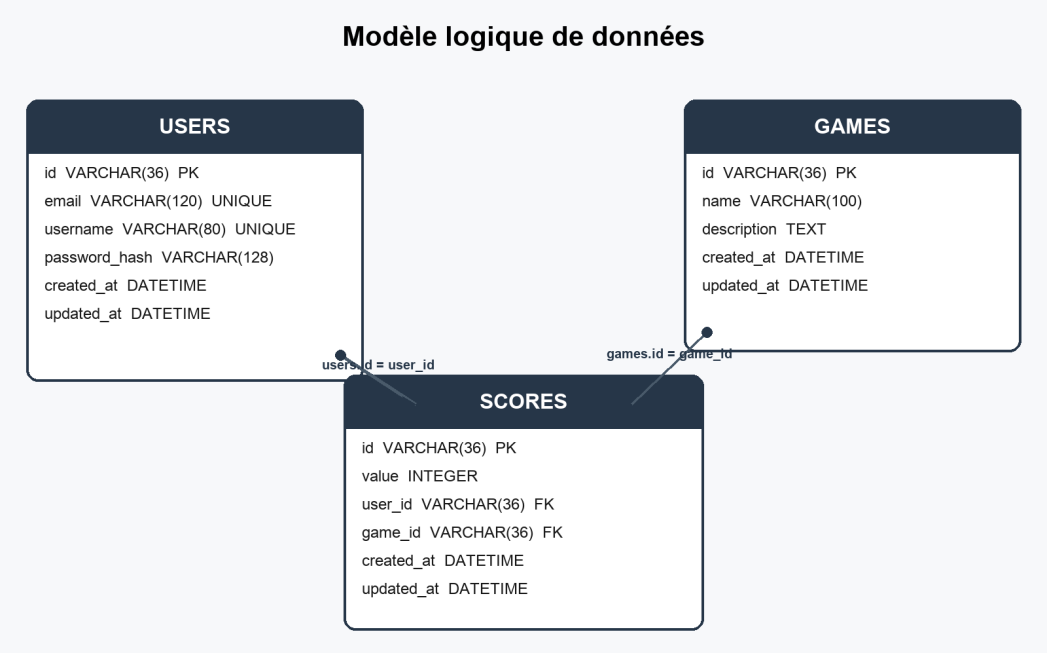
Passé d'un travail d'équipe à une reprise individuelle, OG Game Challenge reflète ma progression du backend vers l'intégration full-stack et la programmation de jeux. Ses limites sont identifiées, mais l'ensemble reste cohérent, fonctionnel et représentatif des compétences que j'ai réellement mises en œuvre.

Annexes illustrées et preuves de réalisation

Les figures suivantes complètent les explications du dossier. Elles montrent le rendu final, l'adaptation aux écrans mobiles, le fonctionnement de l'API et l'exécution des tests automatisés.



Modèle conceptuel Un utilisateur obtient des scores associés à un jeu



Modèle logique Les clés étrangères de SCORES relient USERS et GAMES

Maquette du catalogue sur ordinateur

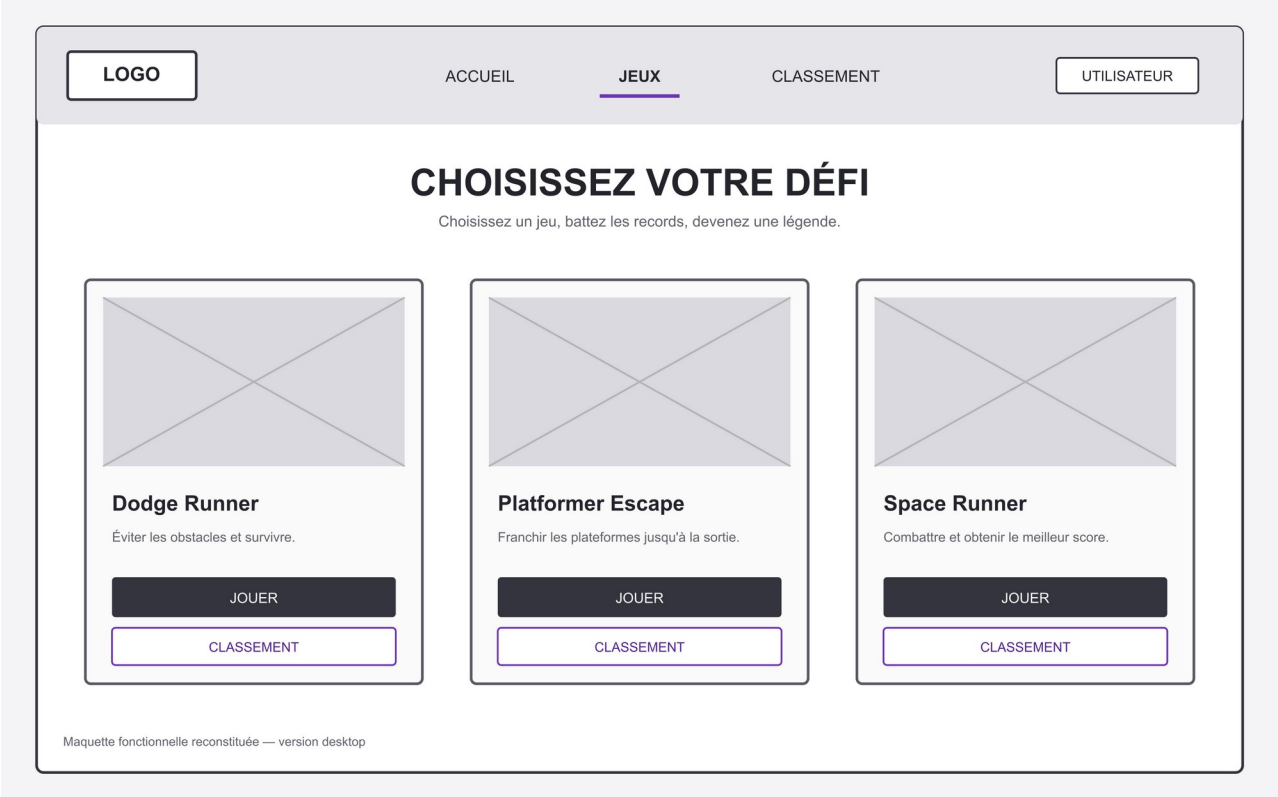


Figure 1 Maquette de conception du catalogue en affichage desktop

Maquette du catalogue sur mobile

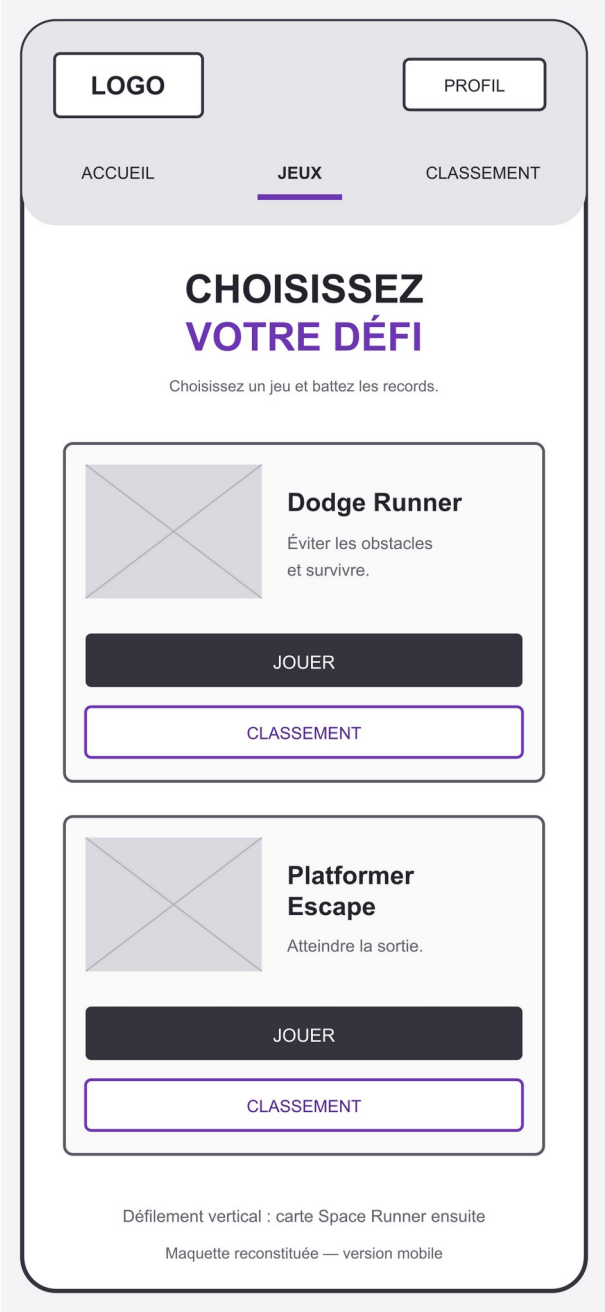


Figure 2 Maquette de conception prévoyant l'adaptation du catalogue sur smartphone

Interface d'accueil

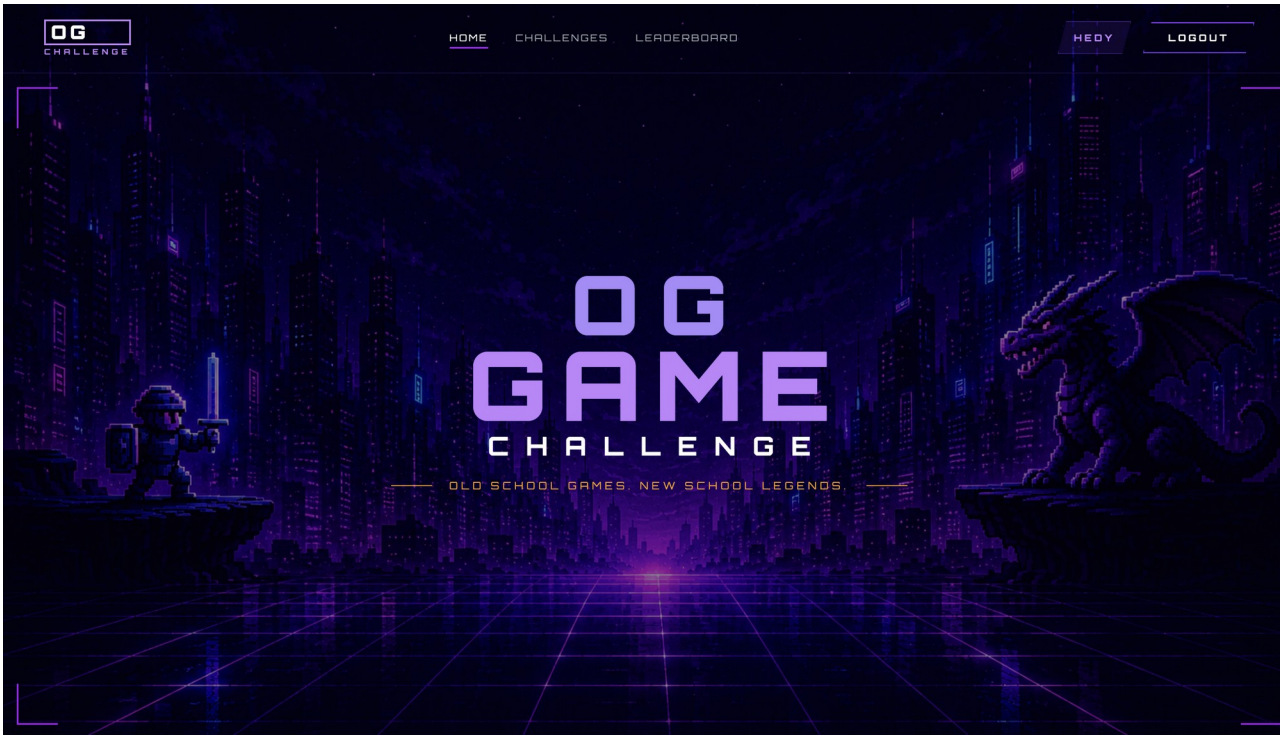


Figure 3 Page d'accueil de l'application après authentification

Catalogue des jeux

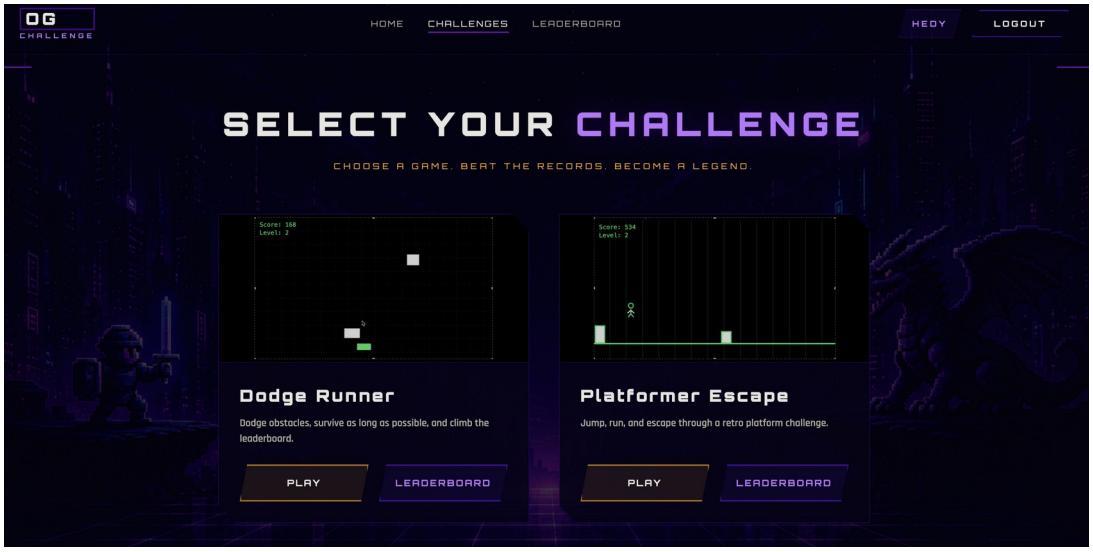


Figure 4 Catalogue présentant Dodge Runner et Platformer Escape

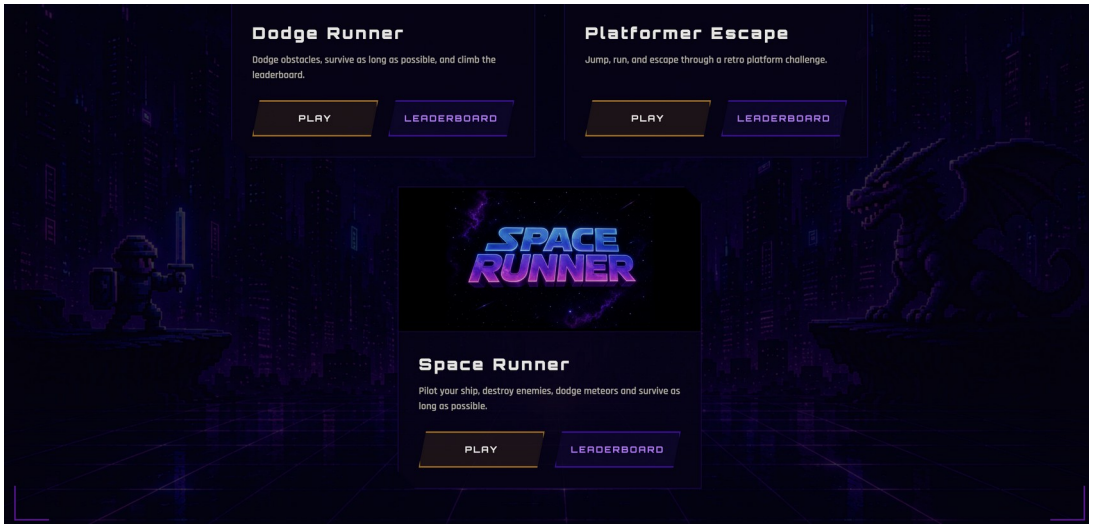


Figure 5 Carte de Space Runner et accès au jeu ou au classement

Classement dynamique

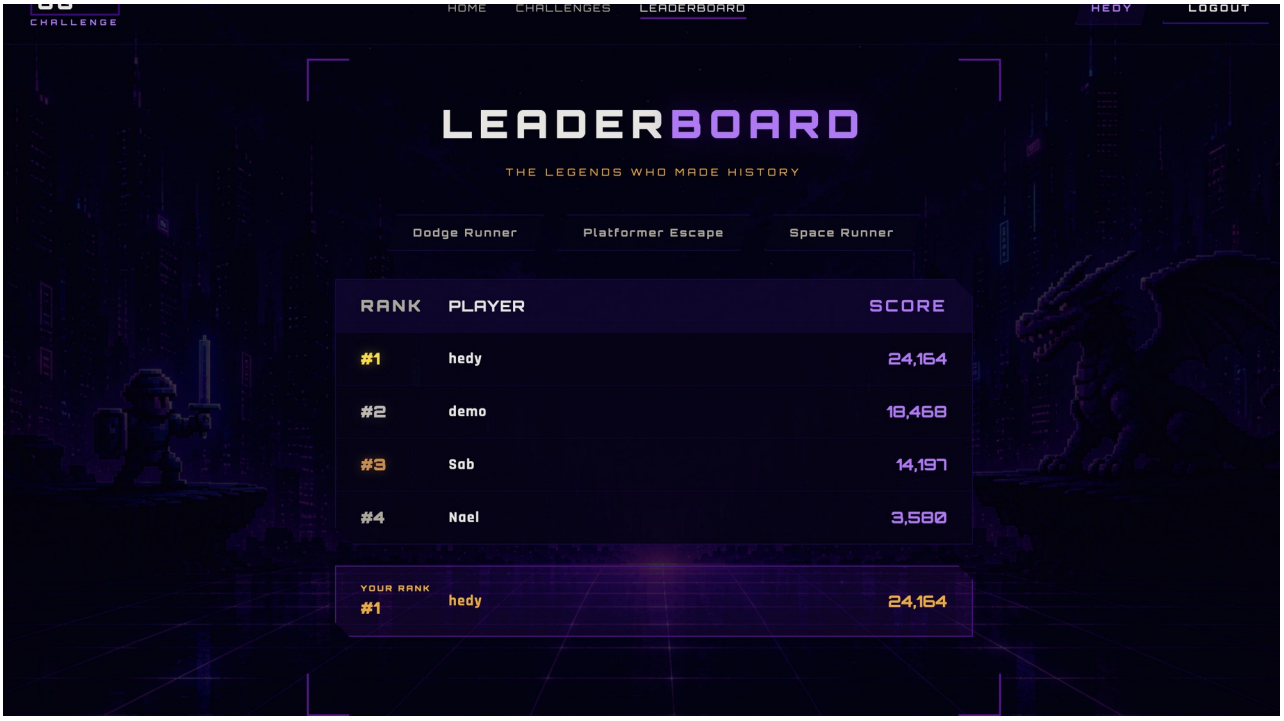


Figure 6 Classement Space Runner alimenté par les scores enregistrés dans la base

Space Runner

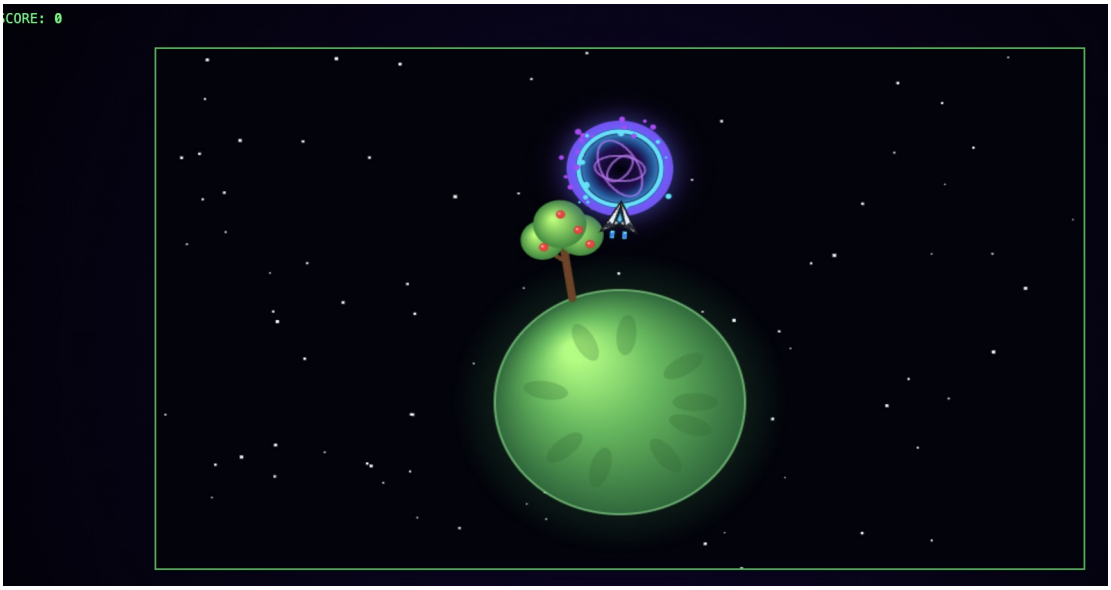


Figure 7 Début de partie et passage par le portail

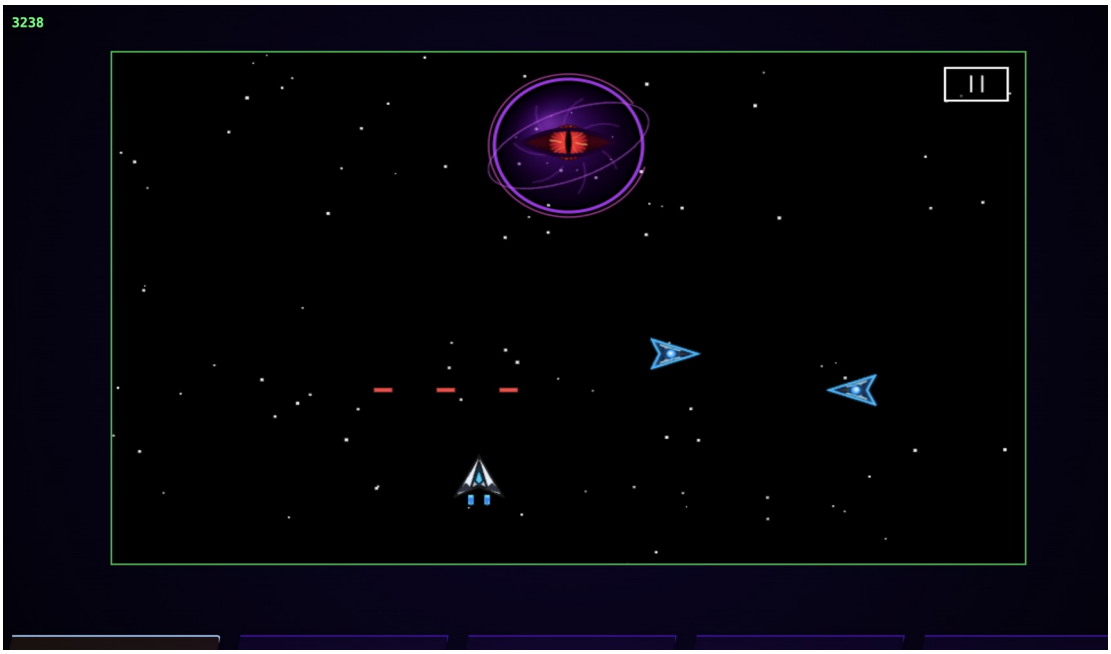


Figure 8 Phase avancée avec ennemis et boss

Adaptation mobile des pages principales

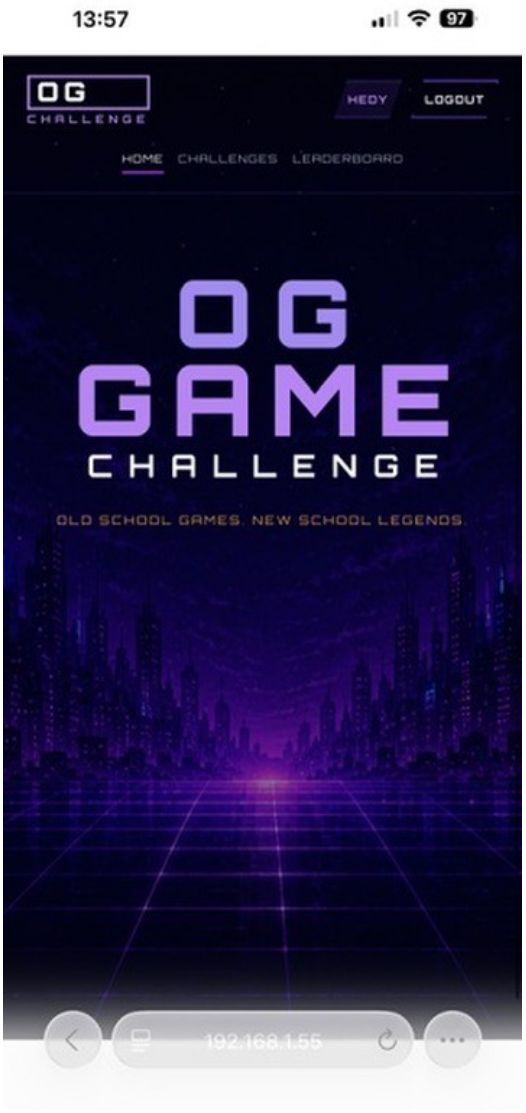


Figure 9 Accueil sur smartphone

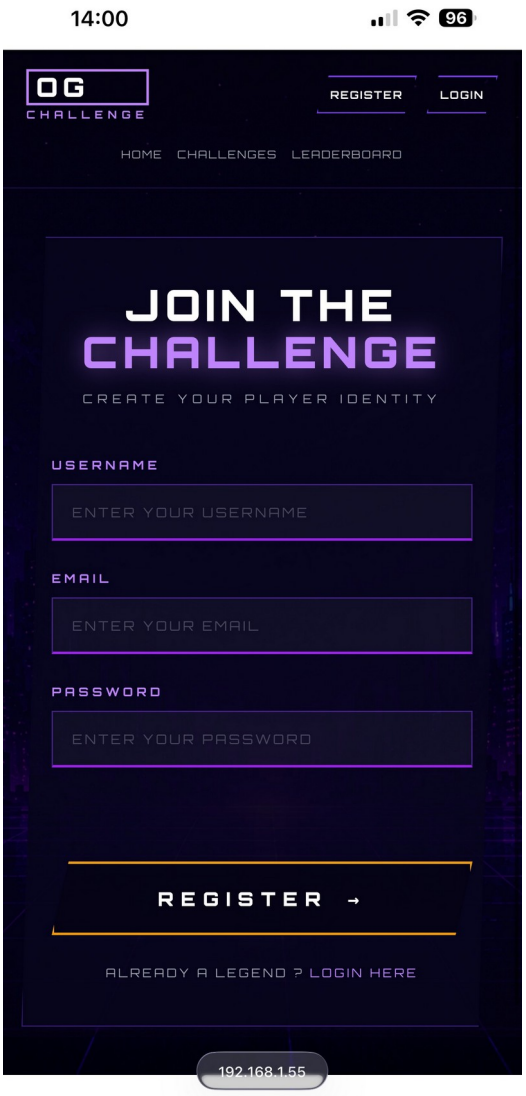


Figure 10 Formulaire d'inscription responsive

Adaptation mobile du contenu dynamique



Figure 11 Catalogue sur smartphone



Figure 12 Classement sur smartphone

Documentation de l'API

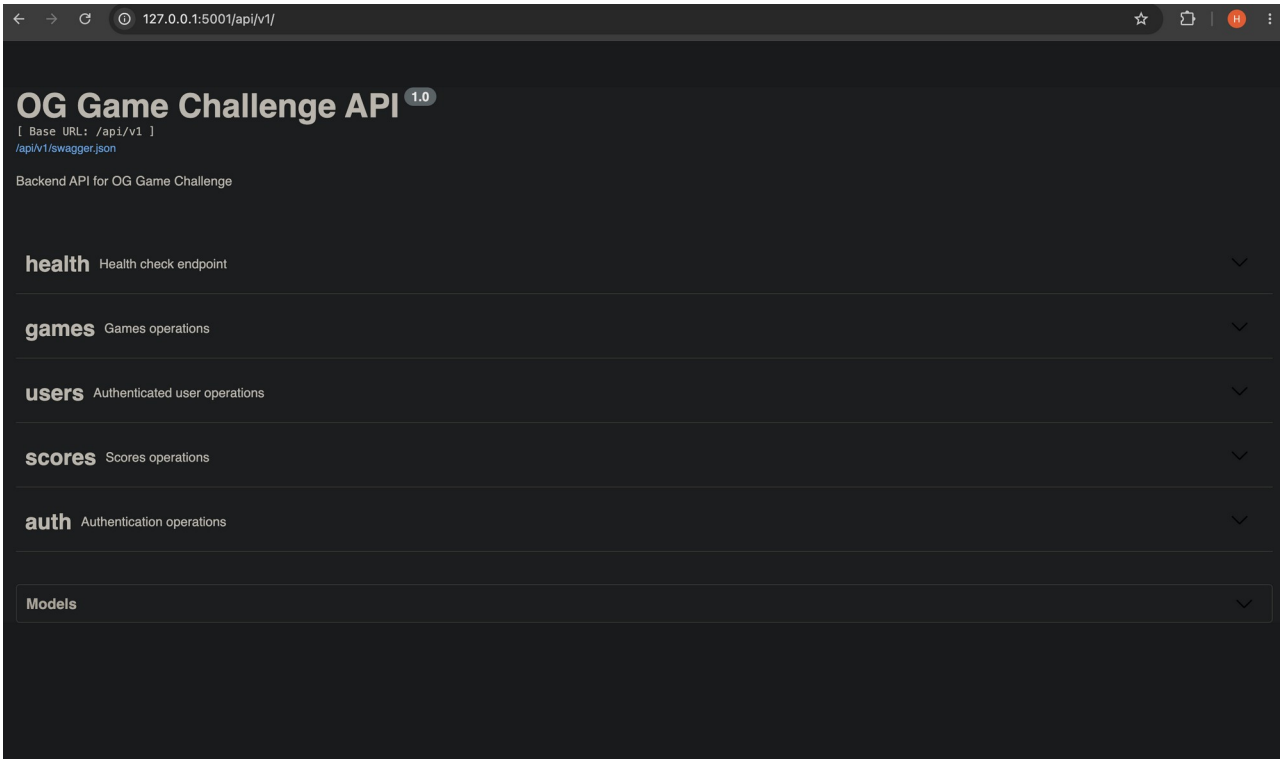


Figure 13 Vue générale des groupes de routes dans Swagger

Création d'un compte par l'API



Figure 14 Inscription réussie avec réponse 201 et données sensibles anonymisées

Lecture d'un jeu par l'API

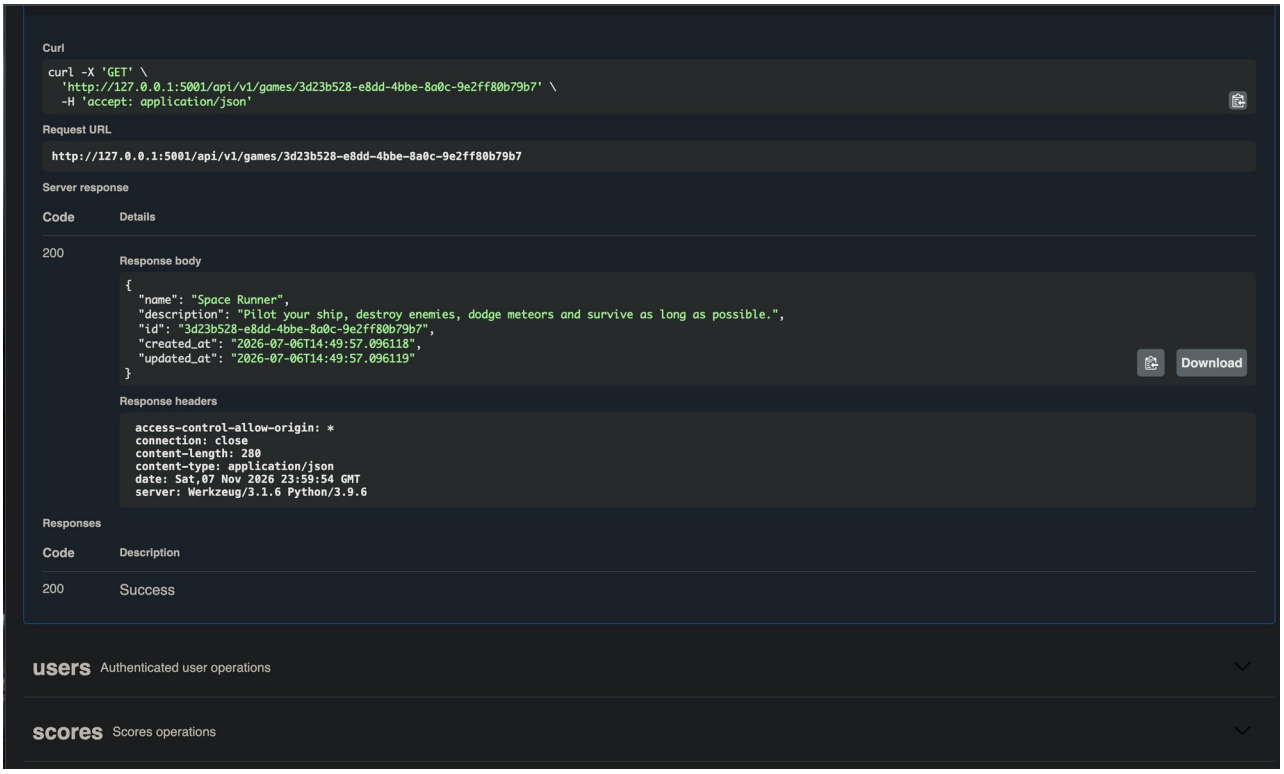


Figure 15 Lecture de Space Runner par son identifiant avec une réponse 200

Exécution des tests automatisés

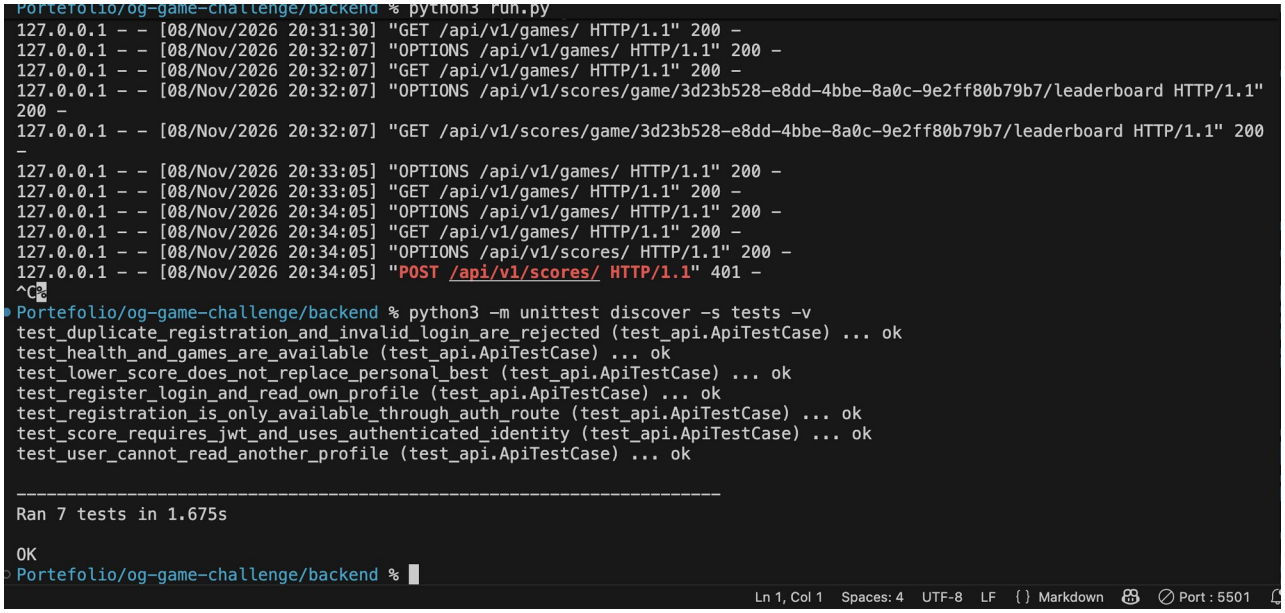


Figure 16 Sept tests d'intégration exécutés avec succès le 8 novembre 2026